DATA 260 HW8 - Farzaan Dalwai

Setup

Repository: https://github.com/farzaandalwai/data260-0686

SID4 0686 | PORT_BASE 8686 | PREFIX s0686 | SEED 686 | VERIFY_SEED 260686 | DOMAIN_ID 6 - Rental Housing Listings

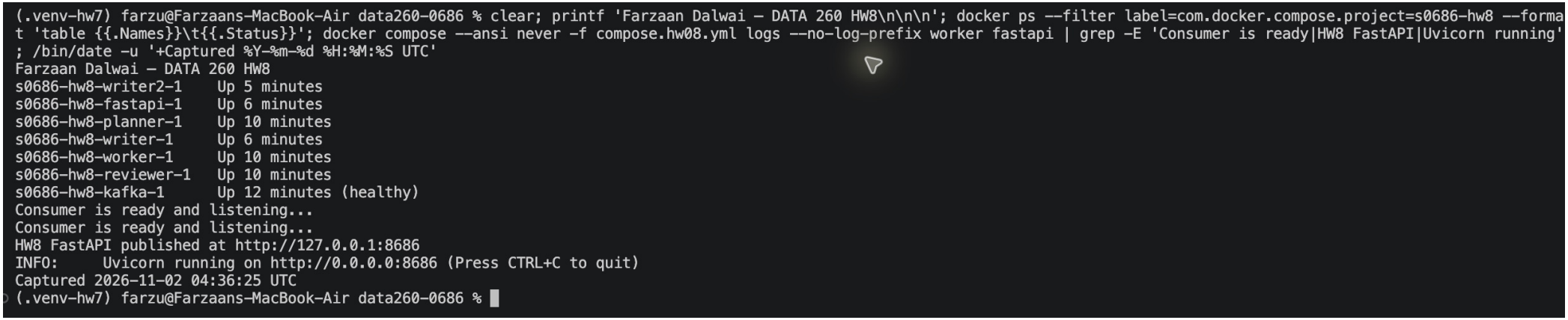
Hardware: Apple M4, 24 GiB RAM, macOS 26.6.2 arm64. Python 3.11.14; Kafka 4.1.2; Docker 29.7.2. Local model: llama3.2:latest via the existing Ollama adapter; failure/order tests use facts-only output.

Tagged commit: hw8 (numeric hash is resolved in the post-tag submission artifact; git rev-parse hw8^{commit}).

GitHub API checked the working public repository and read access for Sbnikitha and supriyaselvanganesan at 2026-11-02 04:42 UTC; no permissions were changed.

hw8_start_api.py, lines 10-11 (excerpt; surrounding lines omitted)

```
    print("HW8 FastAPI published at http://127.0.0.1:8686", flush=True)
    uvicorn.run("main:app", host="0.0.0.0", port=8686)
```



Part 1 - Client and demo

One correlation ID is created for each request, and CREATE's returned UUID is passed to GET.

homework_demo.py, lines 13-16 (excerpt; surrounding lines omitted)

```
    created = request("CREATE_LISTING", SAMPLE)
    assert created["success"], created
    fetched = request("GET_LISTING", {"listing_id": created["listing_id"]})
    assert fetched["success"] and fetched["listing"]["propertyTitle"] == SAMPLE["propertyTitle"]
```

```
(.venv-hw7) farzu@Farzaans-MacBook-Air data260-0686 % clear; printf 'Farzaan Dalwai — DATA 260 HW8\n\n\n'; nl -ba hw8_kafka.py | sed -n '85,96p'; nl -ba homework_demo.py | sed
-n '7,24p'; /bin/date -u '+Captured %Y-%m-%d %H:%M:%S UTC'
Farzaan Dalwai — DATA 260 HW8
    85  def request(operation: str, data: dict, correlation_id: str | None = None) -> dict:
    86      correlation_id = correlation_id or str(uuid4())
    87      reply = consumer(PREFIX + "_client_" + str(uuid4()), RESPONSES)
    88      try:
    89          payload = {"correlation_id": correlation_id, "reply_to": RESPONSES, "operation": operation, "data": data}
    90          log("client_send", topic=REQUESTS, message=payload, correlation_id=correlation_id)
    91          send(producer(), REQUESTS, payload, key=correlation_id)
    92          response = wait_for(reply, correlation_id)
    93          log("client_receive", topic=RESPONSES, message=response, correlation_id=correlation_id)
    94          return response
    95      finally:
    96          reply.close()
     7  SAMPLE = {"propertyTitle": "HW8 San Jose apartment", "location": "San Jose, CA",
     8            "submitterEmail": "hw8@example.com", "description": "A bright two-bedroom rental near campus.",
     9            "propertyType": "Apartment", "termsAccepted": True}
    10
    11
    12  def main():
    13      created = request("CREATE_LISTING", SAMPLE)
    14      assert created["success"], created
    15      fetched = request("GET_LISTING", {"listing_id": created["listing_id"]})
    16      assert fetched["success"] and fetched["listing"]["propertyTitle"] == SAMPLE["propertyTitle"]
    17      time.sleep(0.2)
    18      ids = [created["correlation_id"], fetched["correlation_id"]]
    19      flow = read_events()
    20      flow = [row for row in flow if row.get("correlation_id") in ids]
    21      flow.sort(key=lambda row: (ids.index(row["correlation_id"]), row["timestamp"]))
    22      result = {"create": created, "get": fetched, "flow": flow, "correlation_preserved": all(row["message"]["correlation_id"] == row["correlation_id"] for row in flow if
 "message" in row)}
    23      (RAW / "round_trip.json").write_text(json.dumps(result, indent=2))
    24      print("CREATE_LISTING -> GET_LISTING; correlation IDs preserved:", result["correlation_preserved"])
Captured 2026-11-02 04:42:21 UTC
(.venv-hw7) farzu@Farzaans-MacBook-Air data260-0686 %
```

Part 1 - Worker and API

The worker validates rental fields and stores UUID-keyed listings in a plain dictionary; the existing FastAPI app forwards requests through Kafka.

hw8_worker.py, lines 54-57 (excerpt; surrounding lines omitted)

```
def main():
    listings, completed = {}, {}
    client, output = consumer("s0686_entity_worker", REQUESTS), producer()
    print("Consumer is ready and listening...", flush=True)
```

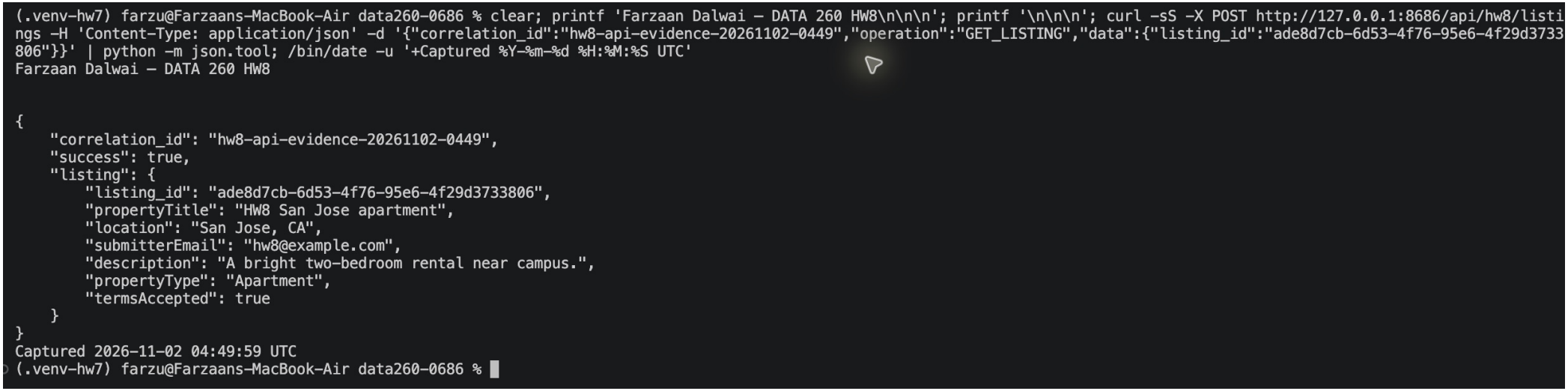
```
(.venv-hw7) farzu@Farzaans-MacBook-Air data260-0686 % clear; printf 'Farzaan Dalwai — DATA 260 HW8\n\n\n'; nl -ba hw8_worker.py | sed -n '27,51p'; nl -ba routers/hw8_listings.p
y | sed -n '16,23p'; /bin/date -u '+Captured %Y-%m-%d %H:%M:%S UTC'
Farzaan Dalwai — DATA 260 HW8
    27  def handle_request(message: dict, listings: dict) -> dict:
    28      cid = message.get("correlation_id")
    29      response = {"correlation_id": cid, "success": False}
    30      data = message.get("data")
    31      if not isinstance(cid, str) or not cid.strip() or message.get("reply_to") != RESPONSES:
    32          return {**response, "error": "Valid correlation_id and reply_to are required"}
    33      if not isinstance(data, dict):
    34          return {**response, "error": "data must be an object"}
    35      if message.get("operation") == "CREATE_LISTING":
    36          try:
    37              fields = ListingData.model_validate(data).model_dump(mode="json")
    38          except ValidationError as error:
    39              fields = [".".join(map(str, item["loc"])) for item in error.errors()]
    40              return {**response, "error": "Invalid listing fields: " + ", ".join(fields)}
    41          listing_id = str(uuid4())
    42          listings[listing_id] = {"listing_id": listing_id, **fields}
    43          return {**response, "success": True, "listing_id": listing_id, "message": "Listing created"}
    44      if message.get("operation") == "GET_LISTING":
    45          try:
    46              UUID(data.get("listing_id", ""))
    47          except (ValueError, TypeError, AttributeError):
    48              return {**response, "error": "A valid listing_id UUID is required"}
    49          listing = listings.get(data["listing_id"])
    50          return {**response, "success": True, "listing": listing} if listing else {**response, "error": "Listing not found"}
    51      return {**response, "error": "Unknown operation"}
    16  @router.post("/listings")
    17  def listing_request(payload: KafkaRequest):
    18      try:
    19          response = request(payload.operation, payload.data, payload.correlation_id)
    20      except (TimeoutError, RuntimeError):
    21          raise HTTPException(status_code=503, detail="Kafka request could not complete") from None
    22      log("api_return", correlation_id=response["correlation_id"], success=response["success"])
    23      return response
Captured 2026-11-02 04:37:15 UTC
(.venv-hw7) farzu@Farzaans-MacBook-Air data260-0686 %
```

Part 1 - API response

The real API GET preserved the caller's correlation ID and returned the listing created by the demo.

routers/hw8_listings.py, lines 16-23 (excerpt; surrounding lines omitted)

```
@router.post("/listings")
def listing_request(payload: KafkaRequest):
    try:
        response = request(payload.operation, payload.data, payload.correlation_id)
    except (TimeoutError, RuntimeError):
        raise HTTPException(status_code=503, detail="Kafka request could not complete") from None
    log("api_return", correlation_id=response["correlation_id"], success=response["success"])
    return response
```



Part 1 - CREATE then GET

Topics: s0686_entity_requests -> s0686_entity_responses; each request has correlation_id, reply_to, operation, data, and each reply echoes correlation_id with success plus listing_id/listing or error.

hw8_kafka.py, lines 85-94 (excerpt; surrounding lines omitted)

```
def request(operation: str, data: dict, correlation_id: str | None = None) -> dict:
    correlation_id = correlation_id or str(uuid4())
    reply = consumer(PREFIX + "_client_" + str(uuid4()), RESPONSES)
    try:
        payload = {"correlation_id": correlation_id, "reply_to": RESPONSES, "operation": operation, "data": data}
        log("client_send", topic=REQUESTS, message=payload, correlation_id=correlation_id)
        send(producer(), REQUESTS, payload, key=correlation_id)
        response = wait_for(reply, correlation_id)
        log("client_receive", topic=RESPONSES, message=response, correlation_id=correlation_id)
        return response
```

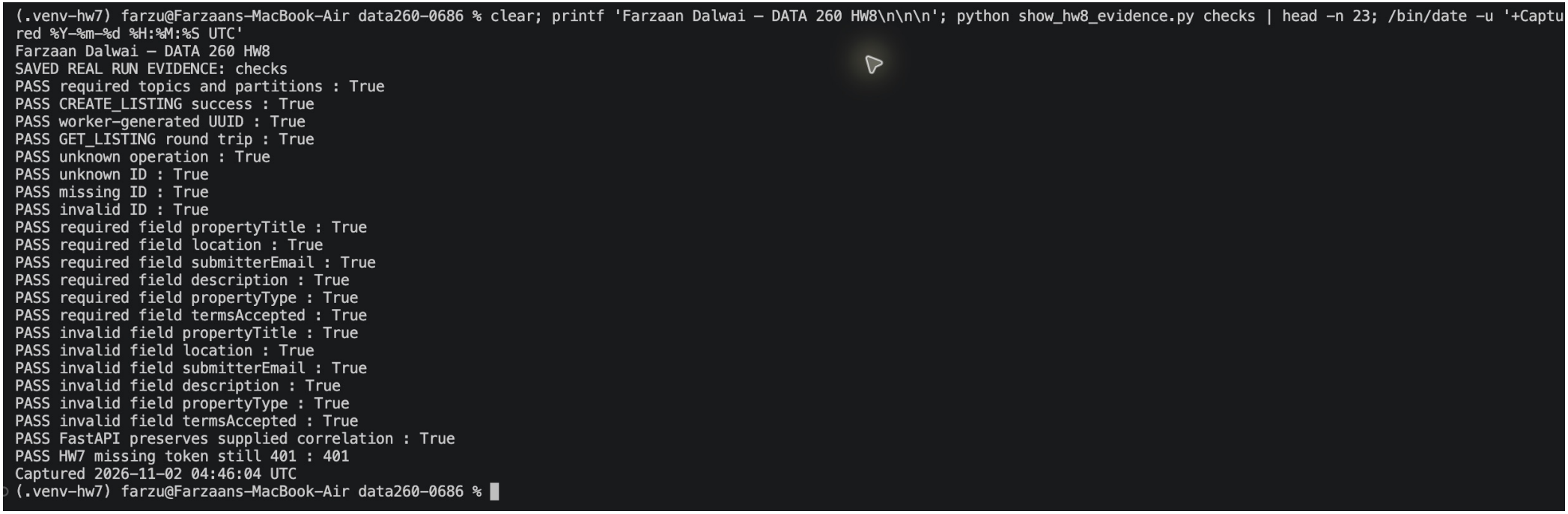
```
(.venv-hw7) farzu@Farzaans-MacBook-Air data260-0686 % clear; printf 'Farzaan Dalwai — DATA 260 HW8\n\n\n'; python show_hw8_evidence.py roundtrip; /bin/date -u '+Captured %Y-%m-
%d %H:%M:%S UTC'
Farzaan Dalwai — DATA 260 HW8
SAVED REAL RUN EVIDENCE: roundtrip
CREATE_LISTING correlation_id=538f4f01-2507-49d6-be11-2de0568017a6
04:39:47.226 client_send 538f4f01-2507-49d6-be11-2de0568017a6 s0686_entity_requests
04:39:48.202 worker_receive 538f4f01-2507-49d6-be11-2de0568017a6
04:39:48.217 worker_process 538f4f01-2507-49d6-be11-2de0568017a6
04:39:48.232 worker_reply 538f4f01-2507-49d6-be11-2de0568017a6 s0686_entity_responses
04:39:48.504 client_receive 538f4f01-2507-49d6-be11-2de0568017a6 s0686_entity_responses
response: {
  "correlation_id": "538f4f01-2507-49d6-be11-2de0568017a6",
  "success": true,
  "listing_id": "ade8d7cb-6d53-4f76-95e6-4f29d3733806",
  "message": "Listing created"
}
GET_LISTING correlation_id=4b8df81f-7a9d-45dd-9f10-a1f317b6c255
04:39:48.546 client_send 4b8df81f-7a9d-45dd-9f10-a1f317b6c255 s0686_entity_requests
04:39:49.122 worker_receive 4b8df81f-7a9d-45dd-9f10-a1f317b6c255
04:39:49.133 worker_process 4b8df81f-7a9d-45dd-9f10-a1f317b6c255
04:39:49.159 worker_reply 4b8df81f-7a9d-45dd-9f10-a1f317b6c255 s0686_entity_responses
04:39:49.244 client_receive 4b8df81f-7a9d-45dd-9f10-a1f317b6c255 s0686_entity_responses
response: {
  "correlation_id": "4b8df81f-7a9d-45dd-9f10-a1f317b6c255",
  "success": true,
  "listing": {
    "listing_id": "ade8d7cb-6d53-4f76-95e6-4f29d3733806",
    "propertyTitle": "HW8 San Jose apartment",
    "location": "San Jose, CA",
    "submitterEmail": "hw8@example.com",
    "description": "A bright two-bedroom rental near campus.",
    "propertyType": "Apartment",
    "termsAccepted": true
  }
}
Captured 2026-11-02 04:41:57 UTC
(.venv-hw7) farzu@Farzaans-MacBook-Air data260-0686 %
```

Part 1 - Validation

Actual checks reject missing or invalid fields, unknown operations, and missing/unknown IDs while preserving the request correlation ID on errors.

hw8_worker.py, lines 10-17 (excerpt; surrounding lines omitted)

```
class ListingData(BaseModel):
    model_config = ConfigDict(extra="forbid", str_strip_whitespace=True)
    propertyTitle: str = Field(min_length=1, max_length=255)
    location: str = Field(min_length=1, max_length=255)
    submitterEmail: EmailStr
    description: str = Field(min_length=26, max_length=2000)
    propertyType: Literal["Apartment", "House", "Condominium", "Townhouse"]
    termsAccepted: Literal[True]
```



Part 2 - Topics

Kafka CLI created/described all required topics; s0686_tasks has two partitions, and the other six have one each.

scripts/hw8_topics.sh, lines 3-12 (excerpt; surrounding lines omitted)

```
for specification in entity_requests:1 entity_responses:1 inbox:1 tasks:2 drafts:1 final:1 dead_letter:1; do
  topic="s0686_${specification%:*}"
  partitions="${specification#*:}"
  docker compose -f compose.hw08.yml exec -T kafka /opt/kafka/bin/kafka-topics.sh \
    --bootstrap-server kafka:19092 --create --if-not-exists --topic "$topic" \
    --partitions "$partitions" --replication-factor 1
done
docker compose -f compose.hw08.yml exec -T kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server kafka:19092 --describe --topic 's0686_.*'
```

```
(.venv-hw7) farzu@Farzaans-MacBook-Air data260-0686 % clear; printf 'Farzaan Dalwai — DATA 260 HW8\n\n\n'; bash scripts/hw8_topics.sh; /bin/date -u '+Captured %Y-%m-%d %H:%M:%S
 UTC'
Farzaan Dalwai — DATA 260 HW8
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but not both.
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but not both.
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but not both.
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but not both.
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but not both.
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but not both.
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but not both.
Topic: s0686_final      TopicId: fToduGi4T06pISwfPlTi8g PartitionCount: 1       ReplicationFactor: 1    Configs: min.insync.replicas=1
        Topic: s0686_final      Partition: 0    Leader: 1       Replicas: 1     Isr: 1  Elr:    LastKnownElr:
Topic: s0686_dead_letter        TopicId: gOheqePMRGCP-SeOcHyYgA PartitionCount: 1       ReplicationFactor: 1    Configs: min.insync.replicas=1
        Topic: s0686_dead_letter        Partition: 0    Leader: 1       Replicas: 1     Isr: 1  Elr:    LastKnownElr:
Topic: s0686_entity_responses   TopicId: xreVMpVfQGuj51idRoncPA PartitionCount: 1       ReplicationFactor: 1    Configs: min.insync.replicas=1
        Topic: s0686_entity_responses   Partition: 0    Leader: 1       Replicas: 1     Isr: 1  Elr:    LastKnownElr:
Topic: s0686_inbox      TopicId: 7K-A2_XrTGGgTTvIwv-KzA PartitionCount: 1       ReplicationFactor: 1    Configs: min.insync.replicas=1
        Topic: s0686_inbox      Partition: 0    Leader: 1       Replicas: 1     Isr: 1  Elr:    LastKnownElr:
Topic: s0686_tasks      TopicId: H5giOIa-QY-mFQGgCSVGsg PartitionCount: 2       ReplicationFactor: 1    Configs: min.insync.replicas=1
        Topic: s0686_tasks      Partition: 0    Leader: 1       Replicas: 1     Isr: 1  Elr:    LastKnownElr:
        Topic: s0686_tasks      Partition: 1    Leader: 1       Replicas: 1     Isr: 1  Elr:    LastKnownElr:
Topic: s0686_drafts     TopicId: BxFHkmoZQOOBM-tNweRhiw PartitionCount: 1       ReplicationFactor: 1    Configs: min.insync.replicas=1
        Topic: s0686_drafts     Partition: 0    Leader: 1       Replicas: 1     Isr: 1  Elr:    LastKnownElr:
Topic: s0686_entity_requests    TopicId: m5o3o_5YQY2ZhQfy9-QAxA PartitionCount: 1       ReplicationFactor: 1    Configs: min.insync.replicas=1
        Topic: s0686_entity_requests    Partition: 0    Leader: 1       Replicas: 1     Isr: 1  Elr:    LastKnownElr:
Captured 2026-11-02 04:36:51 UTC
(.venv-hw7) farzu@Farzaans-MacBook-Air data260-0686 %
```

Part 2 - Separate agents

Planner, Writer, and Reviewer run in distinct container processes; each wrapper consumes its source topic and publishes to the next topic.

hw8_pipeline.py, lines 13-17 (excerpt; surrounding lines omitted)

```
def plan(message: dict) -> dict:
    question = message.get("question")
    if not isinstance(question, str) or not question.strip():
        raise ValueError("question is required")
    return {**message, "plan": ["Read the supplied rental facts", "Explain the facts without inventing details"]}
```

hw8_pipeline.py, lines 34-41 (excerpt; surrounding lines omitted)

```
def review(message: dict) -> dict:
    answer = message.get("draft")
    if not isinstance(answer, str) or not answer.strip():
        raise ValueError("draft is required")
    facts = message.get("facts", {})
    approved = all(str(value).lower() in answer.lower() for value in facts.values())
    return {**message, "status": "approved" if approved else "needs_review", "answer": answer,
            "correlation_id": message["correlation_id"]}
```

```
(.venv-hw7) farzu@Farzaans-MacBook-Air data260-0686 % clear; printf 'Farzaan Dalwai — DATA 260 HW8\n\n\n'; cat planner.py writer.py reviewer.py; for service in planner writer r
eviewer; do docker top s0686-hw8-$service-1 -eo pid,comm,args; done; /bin/date -u '+Captured %Y-%m-%d %H:%M:%S UTC'
Farzaan Dalwai — DATA 260 HW8
from hw8_kafka import INBOX, TASKS
from hw8_pipeline import plan, run_stage


if __name__ == "__main__":
    run_stage("planner", INBOX, TASKS, plan)
from hw8_kafka import DRAFTS, TASKS
from hw8_pipeline import run_stage, write


if __name__ == "__main__":
    run_stage("writer", TASKS, DRAFTS, write)
from hw8_kafka import DRAFTS, FINAL
from hw8_pipeline import review, run_stage

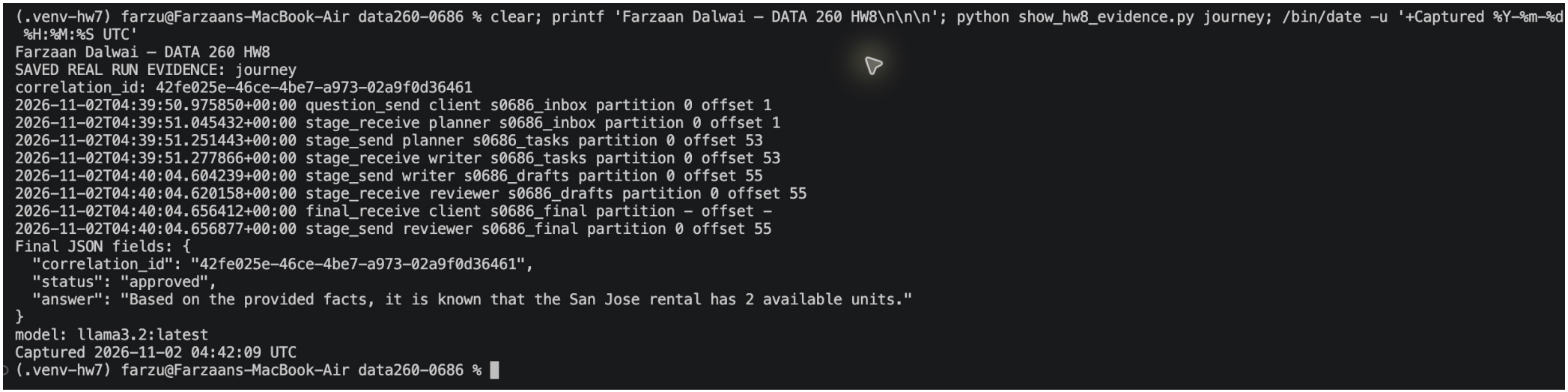

if __name__ == "__main__":
    run_stage("reviewer", DRAFTS, FINAL, review)
PID                 COMMAND             COMMAND
25583               python              python planner.py
PID                 COMMAND             COMMAND
49688               python              python writer.py
PID                 COMMAND             COMMAND
25578               python              python reviewer.py
Captured 2026-11-02 04:37:33 UTC
(.venv-hw7) farzu@Farzaans-MacBook-Air data260-0686 %
```

Part 2 - Message journey

Kafka buffers the hand-offs without direct agent calls: inbox -> Planner -> tasks -> Writer -> drafts -> Reviewer -> final; the final JSON preserves correlation_id, status, and answer.

hw8_pipeline.py, lines 26-31 (excerpt; surrounding lines omitted)

```
    answer = "; ".join(f"{key}: {value}" for key, value in facts.items())
    if message.get("use_model"):
        result = complete([{"role": "user", "content": f"Answer this rental question using only these facts. Keep it short. Question:
{message['question']} Facts: {json.dumps(facts)}"}], temperature=0.0, num_predict=120)
        answer = result["message"]
        message = {**message, "model": result["model"]}
    return {**message, "draft": answer}
```



Part 3 - Duplicate delivery

The unsafe baseline made two effects; replaying the fixed pipeline correlation ID made one draft, and replaying CREATE returned the same UUID.

hw8_pipeline.py, lines 59-63 (excerpt; surrounding lines omitted)

```
        existing = db.execute("SELECT response,sent FROM processed WHERE stage=? AND cid=?", (stage, cid)).fetchone()
        if existing and existing[1]:
            db.commit()
            log("duplicate_skipped", stage=stage, correlation_id=cid, instance=os.getenv("HW8_INSTANCE", stage))
            return False
```

```
(.venv-hw7) farzu@Farzaans-MacBook-Air data260-0686 % clear; printf 'Farzaan Dalwai — DATA 260 HW8\n\n\n'; python show_hw8_evidence.py duplicate; /bin/date -u '+Captured %Y-%m-
%d %H:%M:%S UTC'
Farzaan Dalwai — DATA 260 HW8
SAVED REAL RUN EVIDENCE: duplicate
correlation_id: hw8-e66681cbf8-duplicate
Before fix: side effects = 2
2026-11-02T04:40:10.175163+00:00 baseline_effect effects 1 offset 54
2026-11-02T04:40:10.301866+00:00 baseline_effect effects 2 offset 55
After fix: new drafts= 1 replay skipped= True
Entity replay same generated UUID: True 3bf1ee21-7ec9-4add-8d9d-477b58f80f6b
04:40:16.850462 stage_send {'partition': 0, 'offset': 60, 'topic': 's0686_drafts'}
04:40:16.865327 offset_commit {'partition': 0, 'processed_offset': 60, 'committed_offset': 61, 'topic': 's0686_tasks'}
04:40:16.966883 duplicate_skipped {}
04:40:16.980276 offset_commit {'partition': 0, 'processed_offset': 61, 'committed_offset': 62, 'topic': 's0686_tasks'}
Captured 2026-11-02 04:42:32 UTC
(.venv-hw7) farzu@Farzaans-MacBook-Air data260-0686 %
```

Part 3 - Consumer death

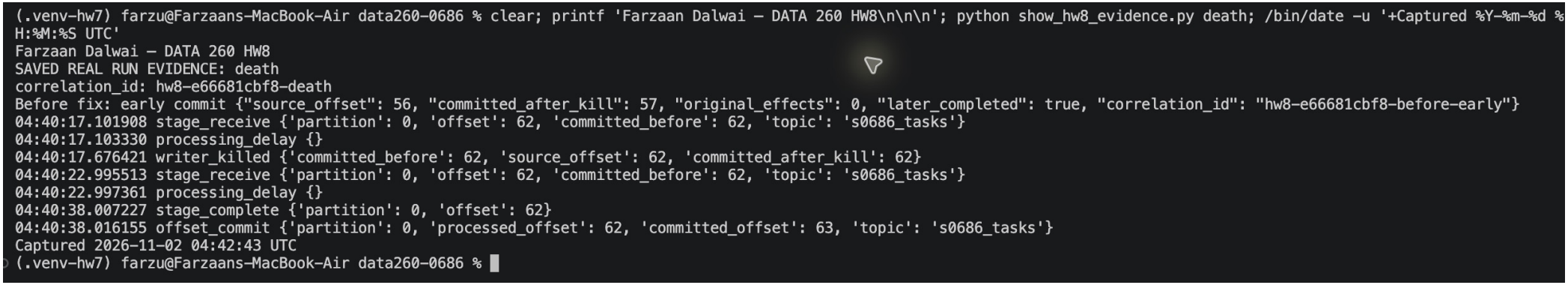
The early-commit baseline lost unfinished work; the fixed Writer received task offset 62 again after SIGKILL and committed 63 only after successful completion.

hw8_pipeline.py, lines 109-115 (excerpt; surrounding lines omitted)

```
            for attempt in range(1, 4):
                try:
                    emitted = process(stage, payload, output, destination, handler, db)
                    log("stage_complete", stage=stage, instance=instance, correlation_id=cid,
                        partition=record.partition(), offset=record.offset(), sequence=payload.get("sequence"),
                        run_id=payload.get("run_id"), emitted=emitted)
                    break
```

hw8_pipeline.py, lines 127-129 (excerpt; surrounding lines omitted)

```
            client.commit(record, asynchronous=False)
            log("offset_commit", stage=stage, instance=instance, correlation_id=cid, topic=source,
                partition=record.partition(), processed_offset=record.offset(), committed_offset=record.offset() + 1)
```



Part 3 - Poison message

The unsafe consumer kept retrying and blocked the next task; the fixed Writer tried three times, published to s0686_dead_letter, and completed the later task.

hw8_pipeline.py, lines 119-126 (excerpt; surrounding lines omitted)

```
                    if attempt == 3:
                        dead = {"correlation_id": cid, "status": "dead_letter", "stage": stage,
                                "attempts": attempt, "error": str(error), "original": payload,
                                "source_topic": source, "partition": record.partition(), "offset": record.offset()}
                        metadata = send(output, DEAD, dead, key=str(cid))
                        log("dead_letter", stage=stage, correlation_id=cid, **metadata, message=dead)
                    else:
                        time.sleep(0.2)
```

```
(.venv-hw7) farzu@Farzaans-MacBook-Air data260-0686 % clear; printf 'Farzaan Dalwai — DATA 260 HW8\n\n\n'; python show_hw8_evidence.py poison; /bin/date -u '+Captured %Y-%m-%d
%H:%M:%S UTC'
Farzaan Dalwai — DATA 260 HW8
SAVED REAL RUN EVIDENCE: poison
correlation_id: hw8-e66681cbf8-poison
Before fix: {"observed_retries_before_kill": 5, "later_completed": false, "correlation_id": "hw8-e66681cbf8-before-poison"}
Actual dead-letter fields: {
  "correlation_id": "hw8-e66681cbf8-poison",
  "status": "dead_letter",
  "stage": "writer",
  "attempts": 3,
  "error": "question is required",
  "source_topic": "s0686_tasks",
  "partition": 0,
  "offset": 63
}
04:40:38.105580 retry {'partition': 0, 'offset': 63, 'attempt': 1}
04:40:38.309377 retry {'partition': 0, 'offset': 63, 'attempt': 2}
04:40:38.511302 retry {'partition': 0, 'offset': 63, 'attempt': 3}
04:40:38.514653 dead_letter {'partition': 0, 'offset': 5, 'topic': 's0686_dead_letter'}
04:40:38.522691 offset_commit {'partition': 0, 'processed_offset': 63, 'committed_offset': 64, 'topic': 's0686_tasks'}
04:40:38.527719 stage_complete {'partition': 0, 'offset': 64}
04:40:38.535572 offset_commit {'partition': 0, 'processed_offset': 64, 'committed_offset': 65, 'topic': 's0686_tasks'}
Captured 2026-11-02 04:42:54 UTC
(.venv-hw7) farzu@Farzaans-MacBook-Air data260-0686 %
```

Part 3 - Unkeyed ordering

The 20 seeded related messages were round-robin distributed without a key; two Writers completed them out of global send order, while each partition retained its own order.

run_hw8_experiments.py, lines 189-192 (excerpt; surrounding lines omitted)

```
    rng = random.Random(260686)
    inputs = [{"sequence": index, "delay_seconds": round(rng.uniform(0.04, 0.25), 3),
               "facts": {"location": "San Jose", "available_units": rng.randint(1, 5)}} for index in range(1, 21)]
    inputs[0]["delay_seconds"] = 0.8
```

```
(.venv-hw7) farzu@Farzaans-MacBook-Air data260-0686 % clear; printf 'Farzaan Dalwai — DATA 260 HW8\n\n\n'; python show_hw8_evidence.py unkeyed; /bin/date -u '+Captured %Y-%m-%d
 %H:%M:%S UTC'
Farzaan Dalwai — DATA 260 HW8
SAVED REAL RUN EVIDENCE: unkeyed
VERIFY_SEED=260686; s0686_tasks: 2 partitions; group=s0686_writer; 2 instances
Key: none (round-robin partitions)
index seq partition offset instance  actual completion timestamp
    1   2         1     10 writer-2  2026-11-02T04:40:41.500138+00:00
    2   4         1     11 writer-2  2026-11-02T04:40:41.725063+00:00
    3   6         1     12 writer-2  2026-11-02T04:40:41.851983+00:00
    4   8         1     13 writer-2  2026-11-02T04:40:42.012369+00:00
    5   1         0     65 writer-1  2026-11-02T04:40:42.060967+00:00
    6  10         1     14 writer-2  2026-11-02T04:40:42.199023+00:00
    7   3         0     66 writer-1  2026-11-02T04:40:42.419337+00:00
    8  12         1     15 writer-2  2026-11-02T04:40:42.421603+00:00
    9   5         0     67 writer-1  2026-11-02T04:40:42.522619+00:00
   10  14         1     16 writer-2  2026-11-02T04:40:42.618727+00:00
   11   7         0     68 writer-1  2026-11-02T04:40:42.767250+00:00
   12  16         1     17 writer-2  2026-11-02T04:40:42.802836+00:00
   13   9         0     69 writer-1  2026-11-02T04:40:42.968135+00:00
   14  18         1     18 writer-2  2026-11-02T04:40:43.052915+00:00
   15  11         0     70 writer-1  2026-11-02T04:40:43.575511+00:00
   16  20         1     19 writer-2  2026-11-02T04:40:43.577398+00:00
   17  13         0     71 writer-1  2026-11-02T04:40:43.759139+00:00
   18  15         0     72 writer-1  2026-11-02T04:40:44.068635+00:00
   19  17         0     73 writer-1  2026-11-02T04:40:44.330625+00:00
   20  19         0     74 writer-1  2026-11-02T04:40:44.720631+00:00
Observed order: [2, 4, 6, 8, 1, 10, 3, 12, 5, 14, 7, 16, 9, 18, 11, 20, 13, 15, 17, 19]
Captured 2026-11-02 04:43:08 UTC
(.venv-hw7) farzu@Farzaans-MacBook-Air data260-0686 %
```

Part 3 - Keyed ordering

The same seeded messages used key rental-0686, all reached partition 0, and completed 1-20; the second Writer stayed in the same group but did not own that entity's partition.

run_hw8_experiments.py, lines 112-117 (excerpt; surrounding lines omitted)

```
    sent = []
    for item in inputs:
        payload = task(f"{run}-{item['sequence']:02}", **item, run_id=run)
        metadata = send(output, TASKS, payload, key="rental-0686" if keyed else None,
                        partition=-1 if keyed else (item["sequence"] - 1) % 2)
        sent.append({**metadata, "sequence": item["sequence"], "correlation_id": payload["correlation_id"], "key": "rental-0686" if keyed else None})
```

```
(.venv-hw7) farzu@Farzaans-MacBook-Air data260-0686 % clear; printf 'Farzaan Dalwai — DATA 260 HW8\n\n\n'; python show_hw8_evidence.py keyed; /bin/date -u '+Captured %Y-%m-%d %
H:%M:%S UTC'
Farzaan Dalwai — DATA 260 HW8
SAVED REAL RUN EVIDENCE: keyed
VERIFY_SEED=260686; s0686_tasks: 2 partitions; group=s0686_writer; 2 instances
Key: rental-0686
index seq partition offset instance  actual completion timestamp
    1   1         0     75 writer-1  2026-11-02T04:40:45.706970+00:00
    2   2         0     76 writer-1  2026-11-02T04:40:46.080904+00:00
    3   3         0     77 writer-1  2026-11-02T04:40:46.378588+00:00
    4   4         0     78 writer-1  2026-11-02T04:40:46.620190+00:00
    5   5         0     79 writer-1  2026-11-02T04:40:46.706737+00:00
    6   6         0     80 writer-1  2026-11-02T04:40:46.834309+00:00
    7   7         0     81 writer-1  2026-11-02T04:40:47.009190+00:00
    8   8         0     82 writer-1  2026-11-02T04:40:47.206530+00:00
    9   9         0     83 writer-1  2026-11-02T04:40:47.373253+00:00
   10  10         0     84 writer-1  2026-11-02T04:40:47.567602+00:00
   11  11         0     85 writer-1  2026-11-02T04:40:47.842440+00:00
   12  12         0     86 writer-1  2026-11-02T04:40:48.007751+00:00
   13  13         0     87 writer-1  2026-11-02T04:40:48.119936+00:00
   14  14         0     88 writer-1  2026-11-02T04:40:48.267337+00:00
   15  15         0     89 writer-1  2026-11-02T04:40:48.408249+00:00
   16  16         0     90 writer-1  2026-11-02T04:40:48.524787+00:00
   17  17         0     91 writer-1  2026-11-02T04:40:48.654293+00:00
   18  18         0     92 writer-1  2026-11-02T04:40:48.801551+00:00
   19  19         0     93 writer-1  2026-11-02T04:40:49.057771+00:00
   20  20         0     94 writer-1  2026-11-02T04:40:49.313651+00:00
Observed order: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
Captured 2026-11-02 04:43:18 UTC
(.venv-hw7) farzu@Farzaans-MacBook-Air data260-0686 %
```

Part 3 - Failure table and HW2 comparison

Failure scenario	Before fix	After fix
Duplicate delivery	2 effects for one correlation ID.	1 draft; replay skipped. CREATE replay returned the same UUID.
Consumer killed mid-work	Early commit 57; source offset 56; 0 original effects.	Offset 62 received twice; committed stayed 62 at kill and became 63 after completion.
Poison message	5 observed retries before stopping; later message blocked.	3 attempts, s0686_dead_letter, later message completed.
Two partitions/two consumers	Unkeyed: 20 completions across partitions 0 and 1; completion order differed from send order.	Entity key rental-0686: all 20 in partition 0, completed 1 through 20.

Two benefits

1. Independent scaling: a second Writer shares the two task partitions without changing Planner or Reviewer. 2. Decoupled failure: a Writer can restart and resume its Kafka input while the other processes remain separate, unlike HW2's waiting function calls in one LangGraph process.

Two responsibilities

1. Handle duplicate delivery and recovery: keep idempotency state, commit only after success, and bound poison retries. 2. Choose partition keys for entity order: Kafka does not provide one global processing order across partitions, so related messages need a shared key.

Limits: the exercise entity dictionary is process-local; the pipeline uses persistent SQLite deduplication, but a publish/mark-sent crash window can still cause downstream duplicates. This is not an end-to-end exactly-once system. Single-broker recreation also loses this local exercise's Kafka data.

Verification

The objective smoke test passed 39/39 checks; React build/lint also passed and all 5,000 existing rental rows remained unchanged.

verify_hw08.py, lines 35-38 (excerpt; surrounding lines omitted)

```
    metadata = AdminClient({"bootstrap.servers": BROKER}).list_topics(timeout=15)
    expected = {REQUESTS: 1, RESPONSES: 1, INBOX: 1, TASKS: 2, DRAFTS: 1, FINAL: 1, DEAD: 1}
    check("required topics and partitions", all(name in metadata.topics and len(metadata.topics[name].partitions) == partitions for name, partitions
in expected.items()))
    created = request("CREATE_LISTING", SAMPLE)
```

```
(.venv-hw7) farzu@Farzaans-MacBook-Air data260-0686 % clear; printf 'Farzaan Dalwai — DATA 260 HW8\n\n\n'; python show_hw8_evidence.py checks | tail -n 19; /bin/date -u '+Captu
red %Y-%m-%d %H:%M:%S UTC'
Farzaan Dalwai — DATA 260 HW8
PASS HW7 missing token still 401 : 401
PASS existing MySQL listings API : True
PASS existing rental rows preserved : 5000
PASS existing MongoDB documents API : True
PASS existing MCP rental search : True
PASS baseline duplicate observed : 2
PASS idempotent replay makes one draft : 1
PASS entity replay preserves UUID : True
PASS killed Writer has not committed its input : True
PASS same offset redelivered : True
PASS commit after restart success : 63
PASS poison retry bound : 3
PASS dead-letter topic and later progress : True
PASS two Writers and two unkeyed partitions : True
PASS twenty unkeyed records : [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
PASS one keyed partition and preserved order : True
PASS three-stage journey with same correlation : ['planner', 'writer', 'reviewer']
PASS final status and answer : True
RESULT: 39 / 39
Captured 2026-11-02 04:43:33 UTC
(.venv-hw7) farzu@Farzaans-MacBook-Air data260-0686 %
```

AI use

1. AI help and my contribution

I used ChatGPT/Codex to read and explain the HW8 instructions, plan the evidence, write the Kafka implementation and verification scripts, debug failures, operate my Mac, and collect the screenshots. I supplied the assignment, repository, and constraints. The assistant performed most of the implementation, execution, and screenshot collection; I am responsible for reviewing the submission.

2. Wrong or unsuitable output

The first implementation assumed concurrent SQLite journal initialization would work and that every line read from a live JSONL file was complete. Real runs exposed a database-lock error and a JSON parsing error. The first startup also exposed transient Kafka topic-metadata errors. An initial regression check expected a list, but the existing MySQL endpoint correctly returned a paginated object; the check was corrected after reading its real implementation.

3. How it was checked

These were detected from the actual container tracebacks and experiment console output. The fixes were checked by rerunning the complete Kafka experiments and the objective smoke checks, comparing offsets, correlation IDs, UUIDs, retry counts, and all 20 ordering records with the saved raw evidence.

4. Changes and why they work

The shared database now uses short transactions without changing journal mode during concurrent startup. The live-log reader waits for a newline before parsing the last record. Separate per-process logs avoid concurrent host/container appends losing records. Consumers continue through metadata errors. The final tests use persistent correlation-ID records for pipeline side effects, manual offset commits after processing, three poison attempts followed by a dead letter, and an entity key for ordered related messages.